SYSTEMS ENGINEERING

A Multi-Agent Approach to Validating Systems Models

Qiana Curcuru
Ethan Cox

June 2026

1951–2026
LINCOLN LABORATORY
MASSACHUSETTS INSTITUTE OF TECHNOLOGY

DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited.This material is based upon work supported by the Department of the Air Force under Air Force Contract No. FA8702-15-D-0001 or FA8702-25-D-B002. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Department of the Air Force.
© 2026 Massachusetts Institute of Technology. Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

Motivation

Forbes



Synopsys Reveals Agentic AI To Speed Next-Gen Chip And System Designs

Synopsys AgentEngineer Brings Multi-Agent AI Workflows To Chip Design. Synopsys is also pushing deeper into AI-assisted semiconductor...

Generative AI Will Be the Next Evolution in Embedded Design



Disruptive technologies, like artificial intelligence (AI), force us to change our behavior. As engineers, AI is reshaping the way we approach research, design, test, and verification. Generative AI, based on large language models (LLMs) and plain text prompts, has a lot to offer here.

Accelerating SDV Architecture with Generative AI-powered MBSE

Generative AI and SysML v2 are unlocking new potential in model-based systems engineering and driving efficient and agile software-defined vehicle...

Sep 9, 2025

Microsoft

MatterGen: A new paradigm of materials design with generative AI

MatterGen enables a new paradigm of generative AI-assisted materials design that allows for efficient exploration of materials, going beyond the limited set of...

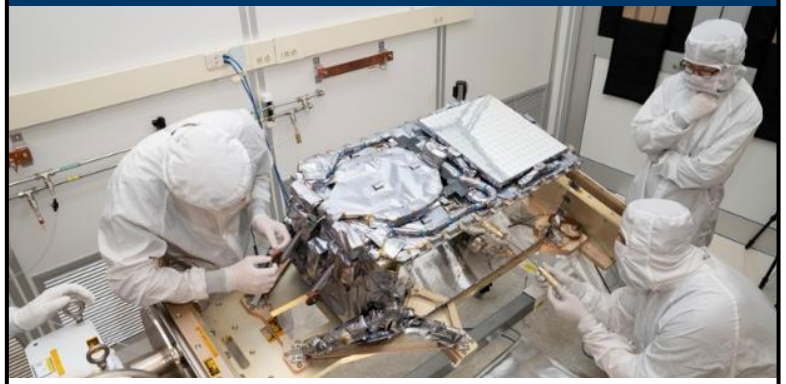
Jan 16, 2025

Everyone's using AI to design systems, but who's making sure they work?

Agentic Verification and Validation

Verification and validation (V&V) of large complex systems can require extensive time-consuming manual review of system requirements and models

Complex



- Programs have 1000+ requirements
- Model V&V requires deep expertise across multiple disciplines

Expensive



- Manual documentation is resource-intensive
- High labor costs for multi-disciplinary review

Time-Consuming



- Iterative development cycles require weeks of preparation
- Identifying gaps early is critical, but time-consuming

Artificial Intelligence (AI) presents an opportunity to make V&V more effective & efficient

MIT Lincoln Laboratory
Who we are

Technology in Support of National Security

MIT Lincoln Laboratory researches and develops a broad array of advanced technologies to meet critical national security needs. What sets us apart from many national R&D laboratories is our focus on building operational prototypes of the unique systems we design.

Key Roles

- System Architecture Engineering
- Long-term Technology Development
- System Prototyping and Demonstration

Technical Divisions

Air, Missile & Maritime Defense Technology	Homeland Protection & Air Traffic Control	Cyber Security and Information Sciences	Communication Systems
Engineering	Advanced Technology	Space Systems and Technology	ISR and Tactical Systems



MIT Lincoln Laboratory
Lexington, Massachusetts

Funding: ~$1B/year

Number of Employees: 4,000

Facilities: 2.1 million sq. ft.

MIT LL does systems engineering on complex systems in a variety of mission areas
AI for systems engineering must secure and reliable

Outline

- **Background and Approach**
- **Agentic Framework**
- **Engineers vs. Agents**
- **Summary and Future Work**

Approach

- Combine expertise of system engineers with modern tooling via agentic AI
- Take advantage of the structured nature of SysML v2
- Use language models to assess requirement flow down and model relationships in an agentic framework



System engineering expertise can be distilled into an agentic assistant for increased efficiency

Background – SysML v2 & Language Models

	Novelty & Advantages	Deployment Challenges
SysML v2	**SysML v2 provides a consistent format to define system requirements and interfaces**	• **Expert, manual, human review is still required** • **Automated validation tools focus on syntax, and cannot interpret semantic meaning**
Language Models (LMs)	• **Process natural language and possess real-world knowledge** • **Can interpret semantic meaning**	• **System models are too large to fit within the conventional context windows of common commercial AI chatbots** • **Commercial AI products cannot be used in proprietary or air-gapped environments** • **Don't possess expertise in systems engineering knowledge**

Language models present opportunities to automate routine, manual review tasks while supporting decision making in system model validation

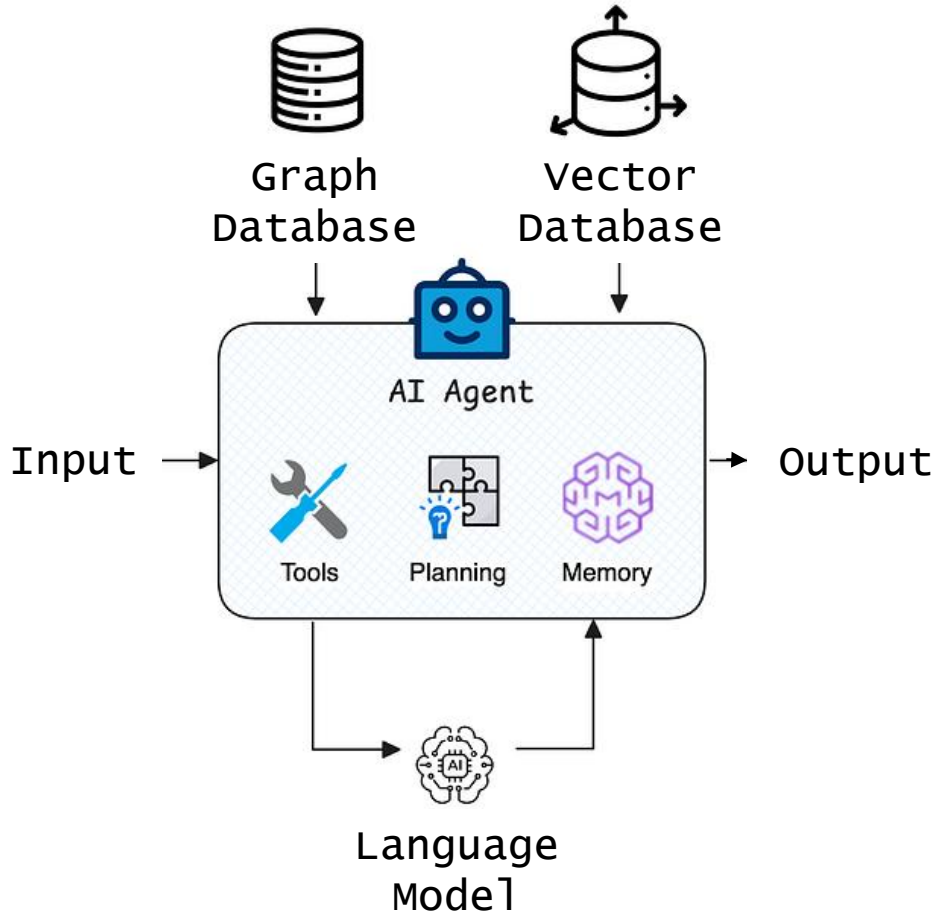
Background – AI Agents

Autonomy

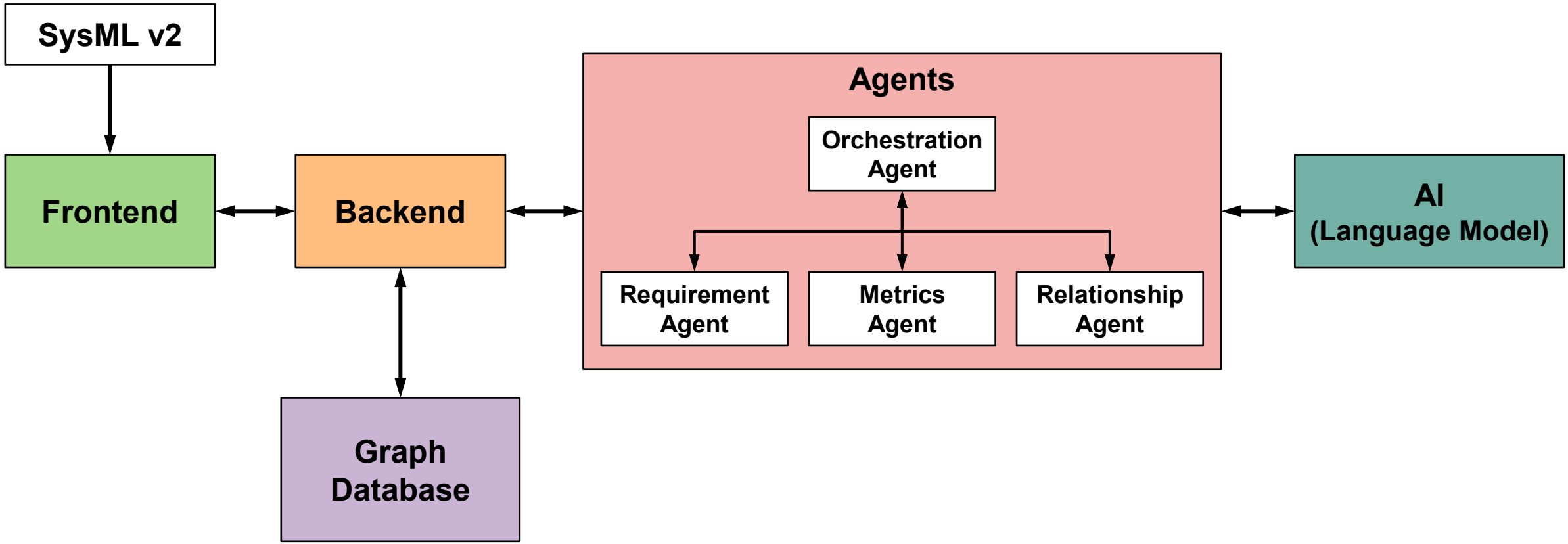
- An *Agent* is a software product that uses AI to pursue goals and **complete tasks** on behalf of users
- Powered by a core **Language Model (LM)** to reason, plan actions, and call tools

Modularity

- Agents have **access to data sources and tools** to achieve specific objectives outside of itself
- **Retrieval Augmented Generation (RAG)** is a technique that enables agents to retrieve new information without additional training
- Easy to extend and scale by plugging in new agents later



Project Overview: Multi-Agent Architecture

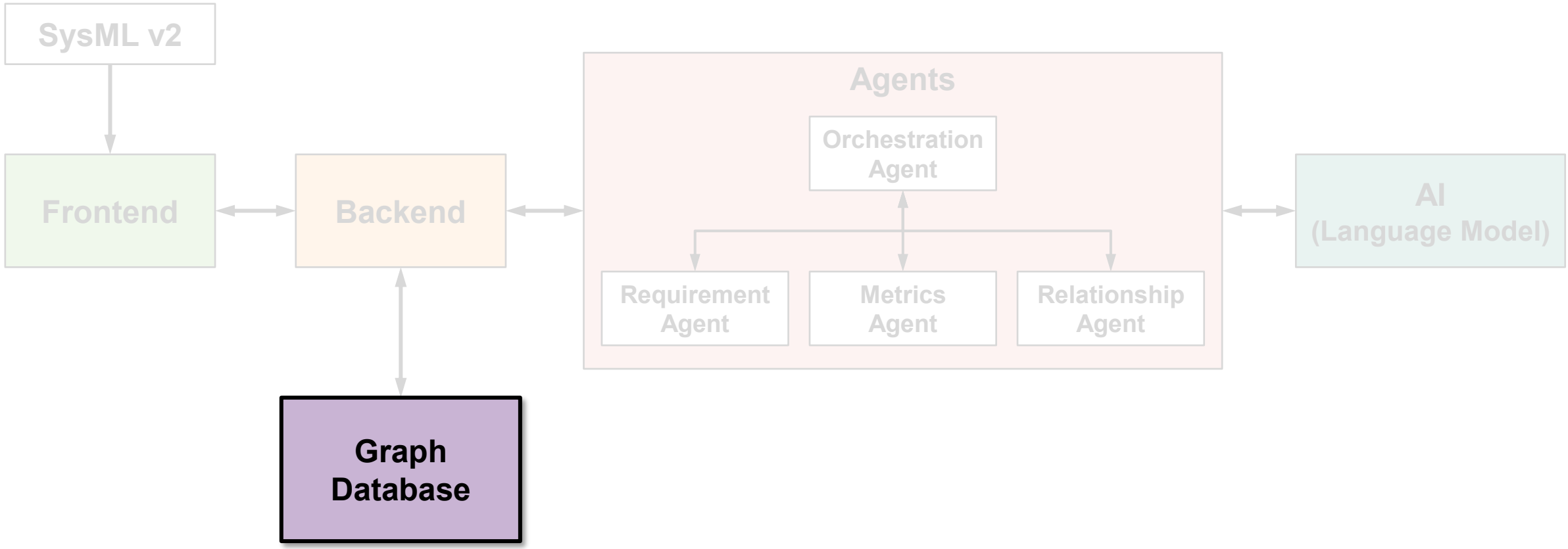


This work creates a model validation assistant using a multi-agentic framework

Outline

- Background and Approach
- **Agentic Framework**
  - **Graph Database**
  - **Multi-Agent Architecture**
  - **AI Implementation**
  - **User interface (Frontend UI)**
  - **Demonstration**
- **Engineers vs. Agents**
- **Summary and Future Work**

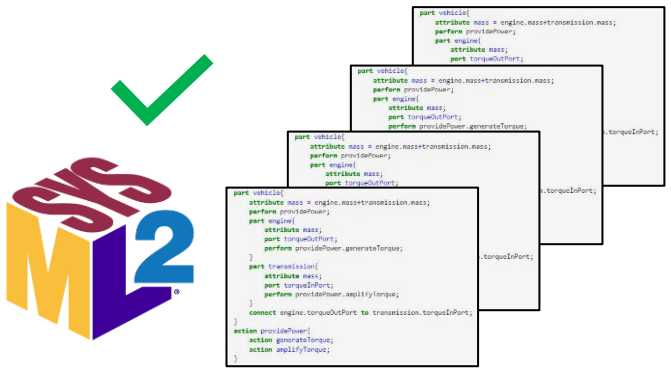
Graph Database



SysML v2 Data Preparation

Source Data

Textual Notation



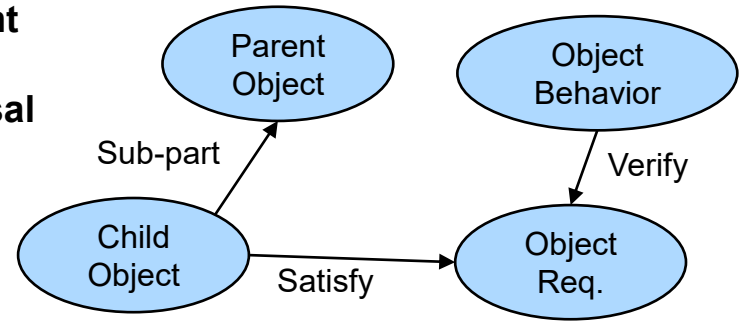
Context Base

- **Serves as the foundation for agent reasoning, retrieval, and validation queries**
- **Normalized model elements in a common schema for uniform reasoning stored in a graph database**

Database Management

Store SysMLv2 models as dynamic subgraphs to enable structural-based context retrieval for agents

Efficient Graph Traversal



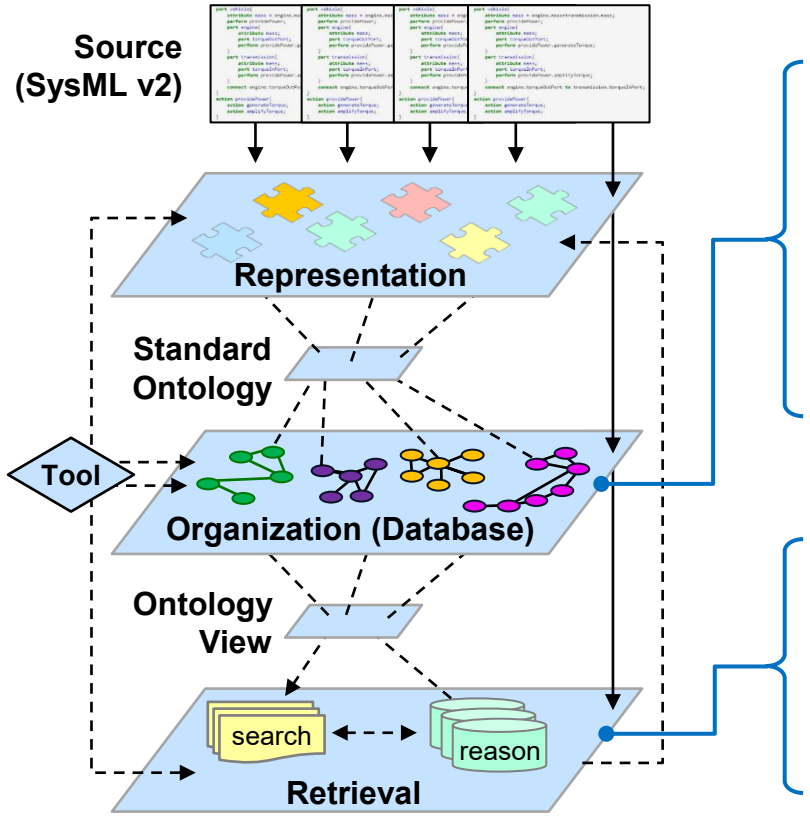
- **Edges: relationships**
  - **Satisfy, Verify, etc.**
- **Nodes: element definitions & usages**
  - **Requirements, Connections**
  - **Parts, Attributes, Ports, Actions**
  - **States, Constraints, Views**

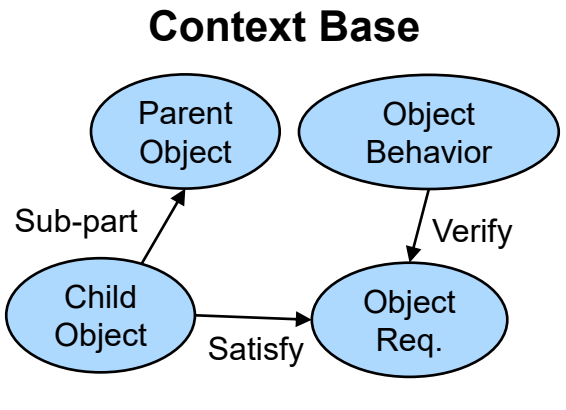
Graph databases allow large system models to be broken into smaller digestible chunks

Graph Retrieval Augmented Generation (GraphRAG)

Data Transformation Methodology

GraphRAG enhances LMs with external system model information





Multi-Agent Reasoning

Agents

- Requirements Agent
- Metrics Agent
- Relationship Agent

Semantic Search

Query

"Find power requirements" → [0.23, -0.15, …]

Req

REQ-PWR-01 → [0.22, -0.14, …]

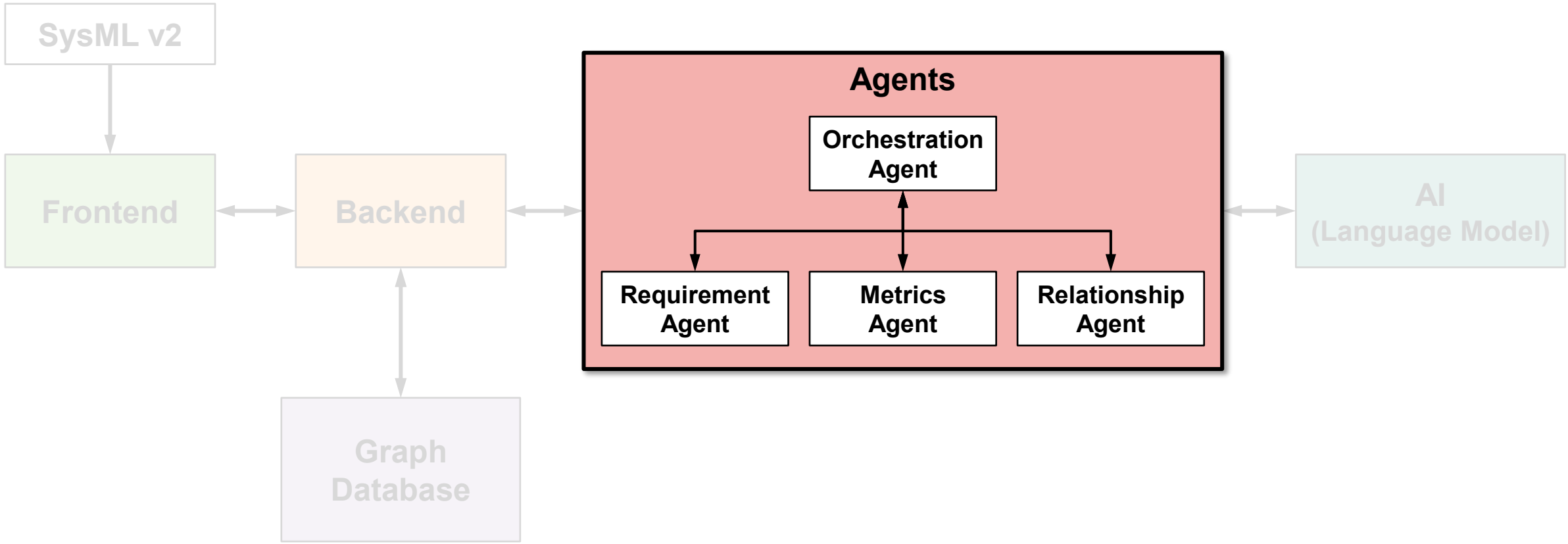
REQ-ME-02 → [0.15, -0.42, …]

$$\overrightarrow{Query} \cdot \overrightarrow{Req} = |Query||Req| \cos\theta$$

Requirement	Dot prod	Result
REQ-PWR-01	0.95	✓
REQ-ME-02	0.62	✗

LINCOLN LABORATORY
MASSACHUSETTS INSTITUTE OF TECHNOLOGY

Project Framework: Multi-Agent Architecture



Test Case Agents

Systems engineering tasks were broken down into two distinct objectives

Requirement Task

Identify the best parent requirement for orphan requirements

- Engineer may not always define a parent for each requirement
- Select the best parent for each orphan requirement using requirement context

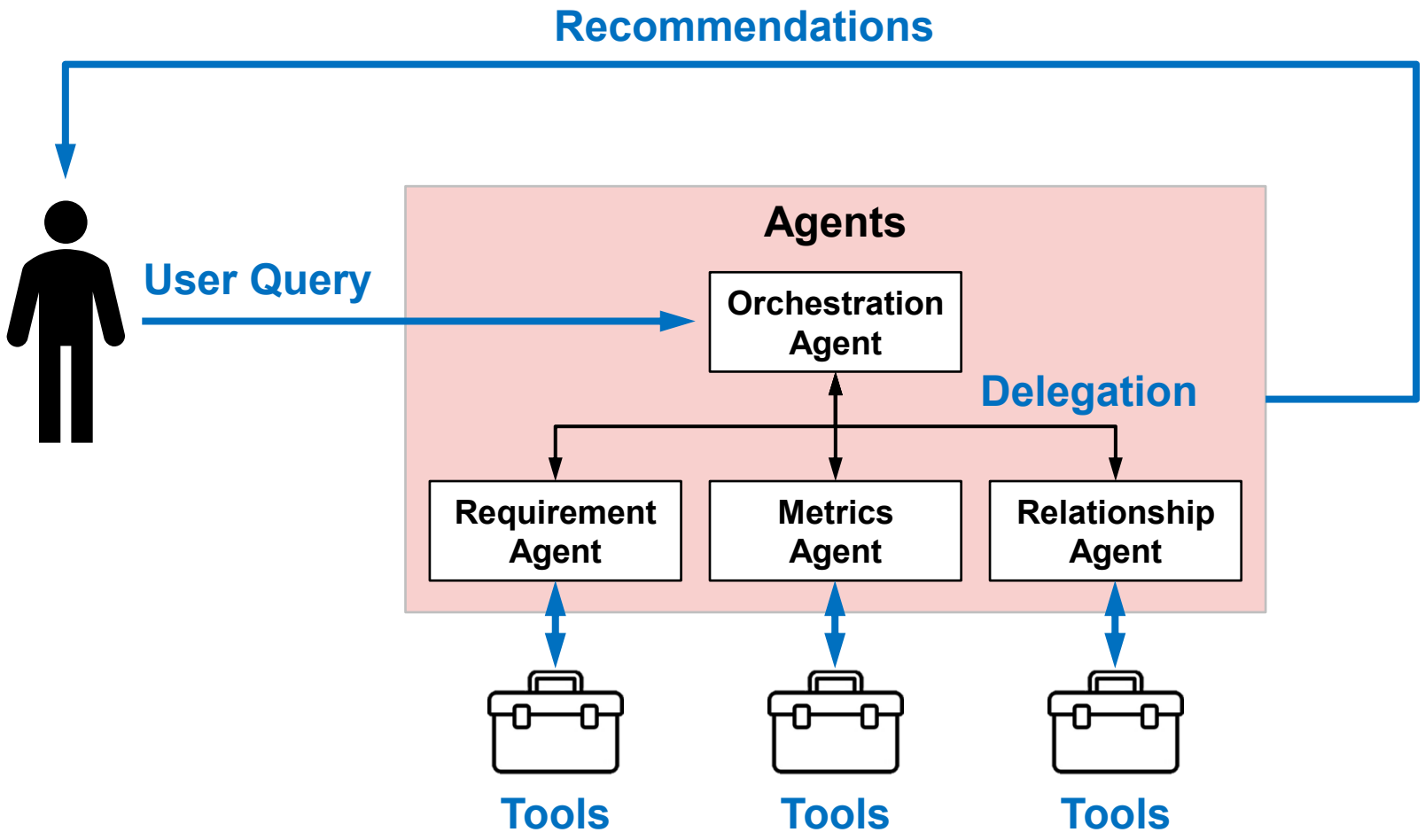
Relationship Task

Find potential missing relationships between requirements and parts

- A requirement may have multiple relationships to parts
- Not all parts need to connect to a requirement

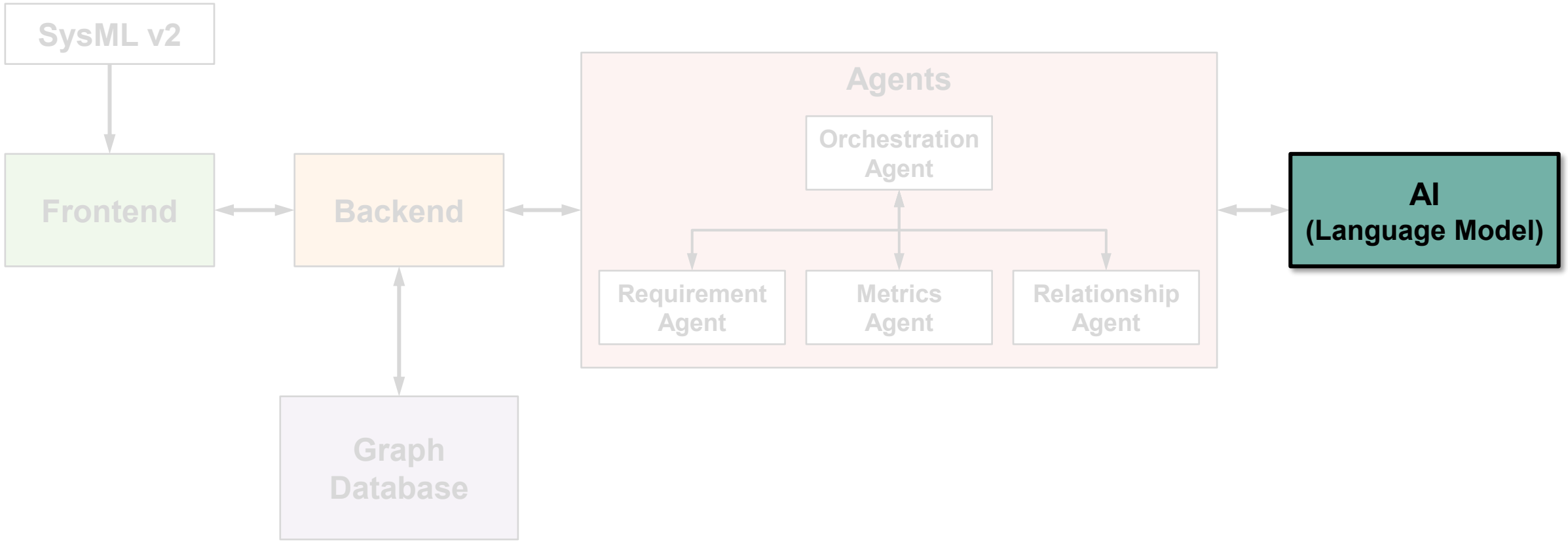
The semantic nature of these tasks make them a good fit for AI agents

Multi-Agent Pipeline



Agents work together to delegate tasks based on which agent would be best suited for the assignment

Deployment


SysML v2
Frontend
Backend
Graph Database
Agents
Orchestration Agent
Requirement Agent
Metrics Agent
Relationship Agent
AI (Language Model)

Small Language Models (SLMs)

SLMs are lightweight language models designed to run efficiently on local or constrained hardware using very few resources (1B-15B parameters or smaller)

- **Capable of reasoning, classification, planning, and transformation tasks**
- **Potential to fine-tuned for domain-specific workflows rather than general-purpose, open-ended dialogue**
- **Great for semantic reasoning**

Powerful Enough

SLMs are powerful enough for many agentic tasks

Can make repeated calls across many model elements, iterating over thousands of requirements

Modifiable

SLMs are easy to fine-tune

Possible to fine-tune to a SysML v2 glossary, project examples, enterprise models, or domain rules

Efficient

SLMs offer fast inference times and low latency

- High speed is crucial for real-time applications like a validator chatbot
- Lowers energy consumption

AI Implementation

Model

- **Model: Mistral 7B**
- **Size: 7 billion parameter model**
- **Performance: top performer on many common LLM benchmarks**

Security

- **Data Control: SLMs can be hosted locally due to their small size**
- **Allows for data privacy and security in proprietary and air-gapped environments**



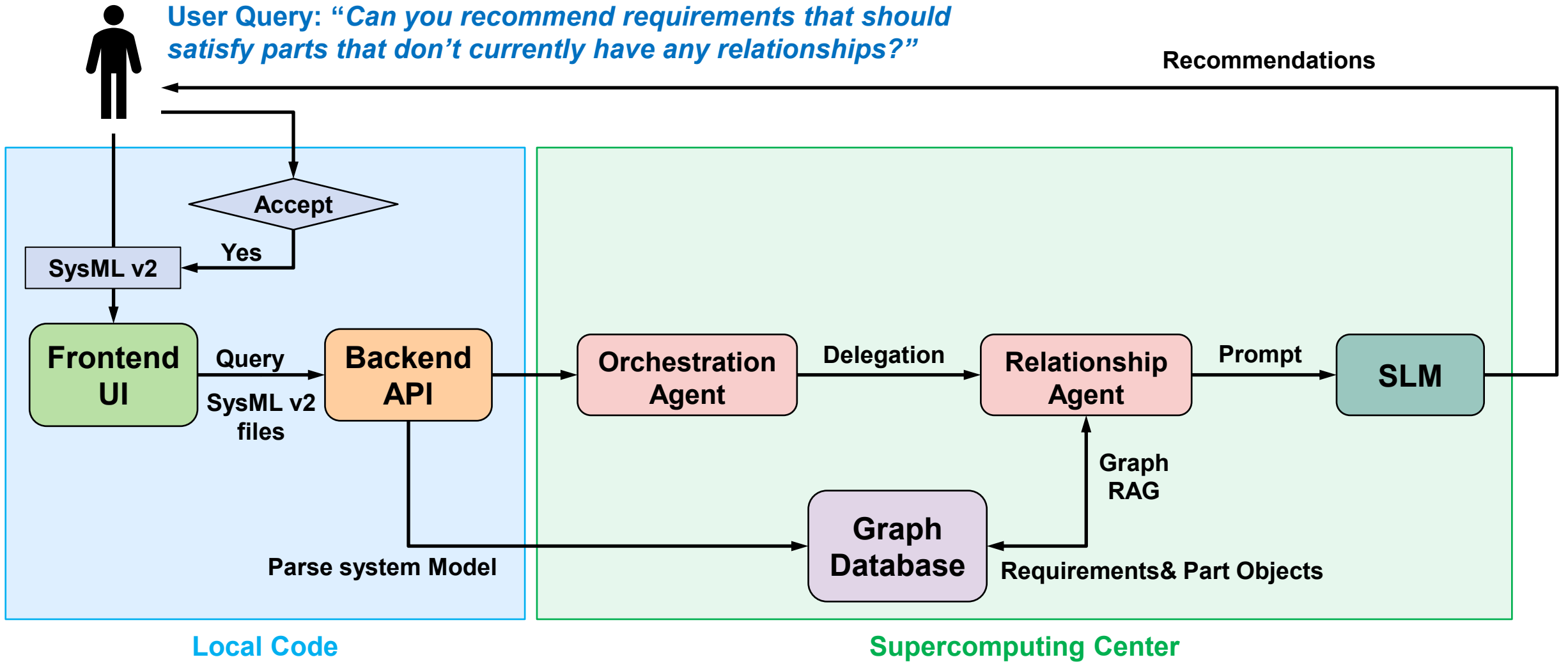
Hardware

- **This work was performed on a supercomputer (contains AI models approved for proprietary data)**
- **Was also tested on a conventional laptop** (32GB RAM)



SLMs are small enough that they can be hosted on a local network minimizing footprint without compromising speed or performance

Project Flow Diagram



User Interface (Frontend UI)

MBSE Chatbot

Hello there!

Advanced Metric Result

User Query	Value
query	Hello there!
result	Greetings to you! This SysMLv2 model contains a total of 16 requirements and 9 objects.

Ask about your model...

Send

Start New Session

SysML Viewer

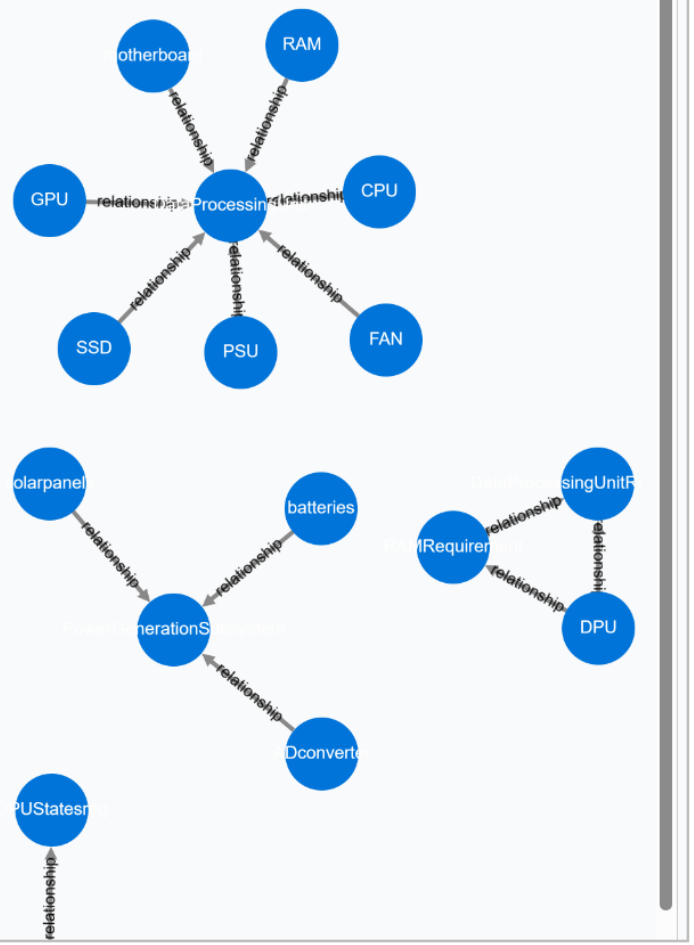
Choose Files 6 files

architecture

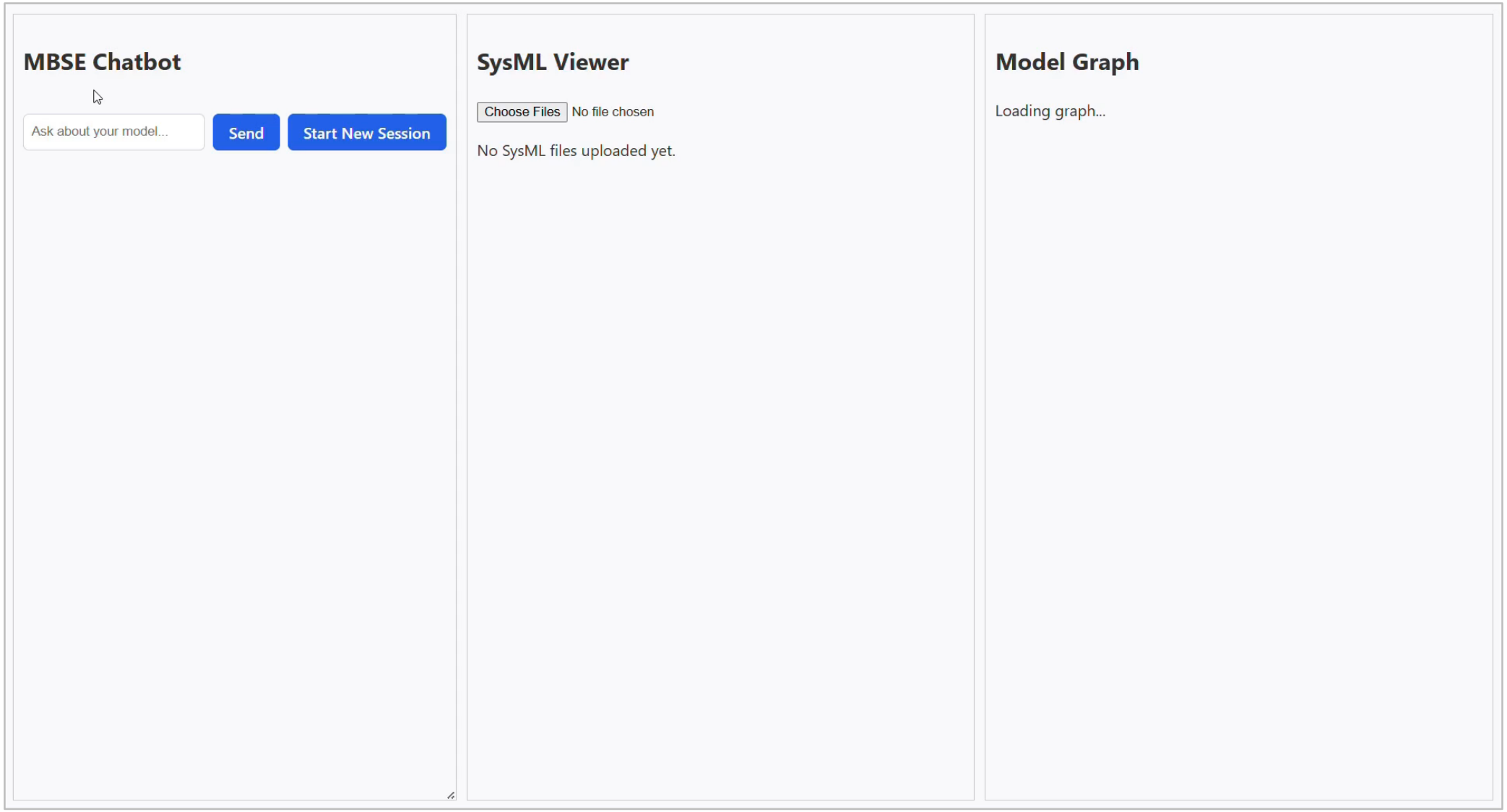
```
package architecture {
  private import ScalarValues::*;
  private import SysML::*;
  private import requirements;
  private import DPU::*;
  private import PowerSub::*;
  private import ISQ::*;

  part def DataProcessingUnit {
    attribute Mass;
  }
  part def PowerGenerationSubsystem{
    attribute Mass;
  }
  part def FlightSysDef{
    attribute mass: ISQ::MassValue;
  }
  part def ThermalManagementSubsystem{
    attribute Mass;
    attribute Temp;
  }
  part def PropulsionSubsystem{
    attribute Mass;
  }
  part def AttitudeNavControl{
    attribute Mass;
    attribute accuracy;
  }
  part CubeSat {
    part DPU : DataProcessingUnit;
    part PowerSub : PowerGenerationSubsystem;
```

Model Graph



User Experience Video



Results - Model Changes

```
part satisfactionContext {
  ref :>> CubeSat;

  satisfy requirement REQ_008 : DataProcessingUnitRequirement by CubeSat.DPU;
  satisfy requirement REQ_009 : RAMRequirement by CubeSat.DPU;

  #derivation connection {
    end r1 ::> REQ_008;
    end r1_1 ::> REQ_009;
  }
}
```

```
part satisfactionContext {
  ref :>> CubeSat;
  ref :>> DataProcessingUnit;
  ref :>> PowerGenerationSubsystem;

  satisfy requirement REQ_008 : DataProcessingUnitRequirement by CubeSat.DPU;
  satisfy requirement REQ_009 : RAMRequirement by CubeSat.DPU;
  satisfy requirement REQ_010 : DPUStatesreq by DataProcessingUnit.GPU;
  satisfy requirement REQ_011 : SSDRequirement by DataProcessingUnit.storage;
  satisfy requirement REQ_012 : DPUStatesreq by PowerGenerationSubsystem.batteries;

  #derivation connection {
    end r1 ::> REQ_008;
    end r1_1 ::> REQ_009;
  }
}
```

Selected Model changes recommended by AI can be implemented in SysML v2 files

Outline

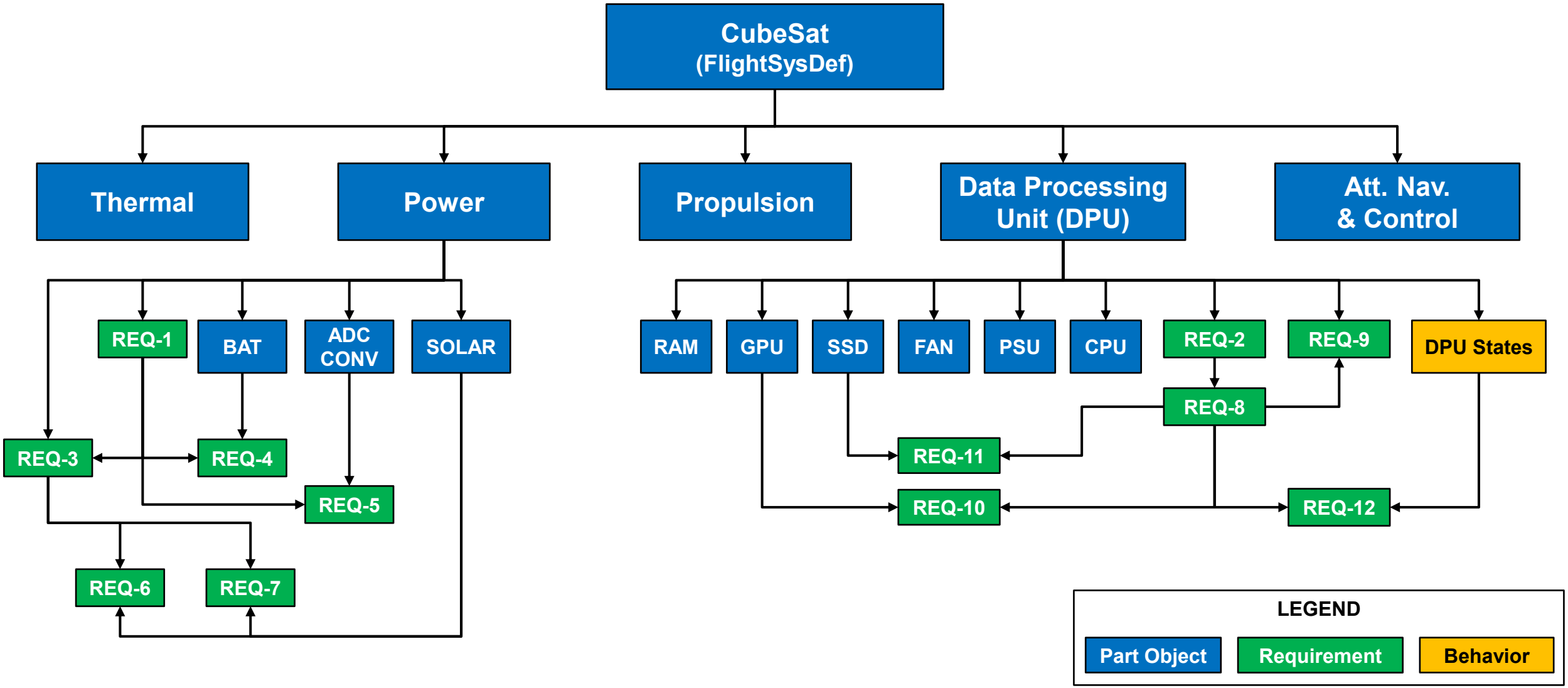
- Background and Approach
- Agentic Framework
- **Engineers vs. Agents**
  - **Experiment Overview**
  - **Results**
- **Summary and Future Work**

Experiment Overview

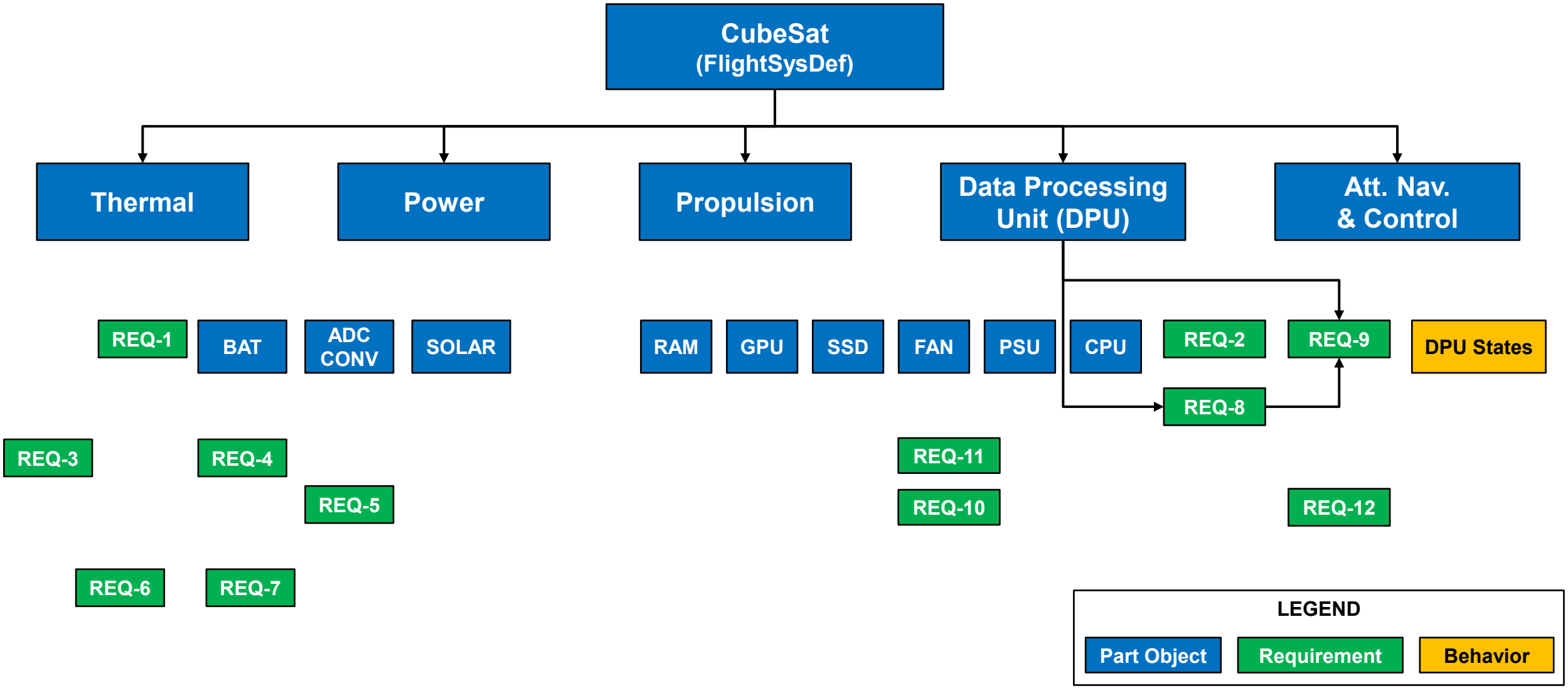
- Agents were presented with a simplified SysML v2 CubeSat model consisting of parts and metadata
- Each test consisted of three rounds: 4, 8, & 12 requirement sets
- The agents were tasked with two tests:
  - Provide the parent requirement for each requirement in each set (requirement agent).
  - Provide the relevant subsystem(s) that relate to each requirement (relationship agent)
- The agent's results were compared with four human Systems Engineers at MIT Lincoln Laboratory
  - Two senior level engineers (>10 years of professional experience)
  - Two junior level engineers (<5 years of professional experience)

This experiment will compare the agents' speed and correctness against four systems engineers

Simplified CubeSat Model
Correct Model



Simplified CubeSat Model Missing Relationships



Requirements

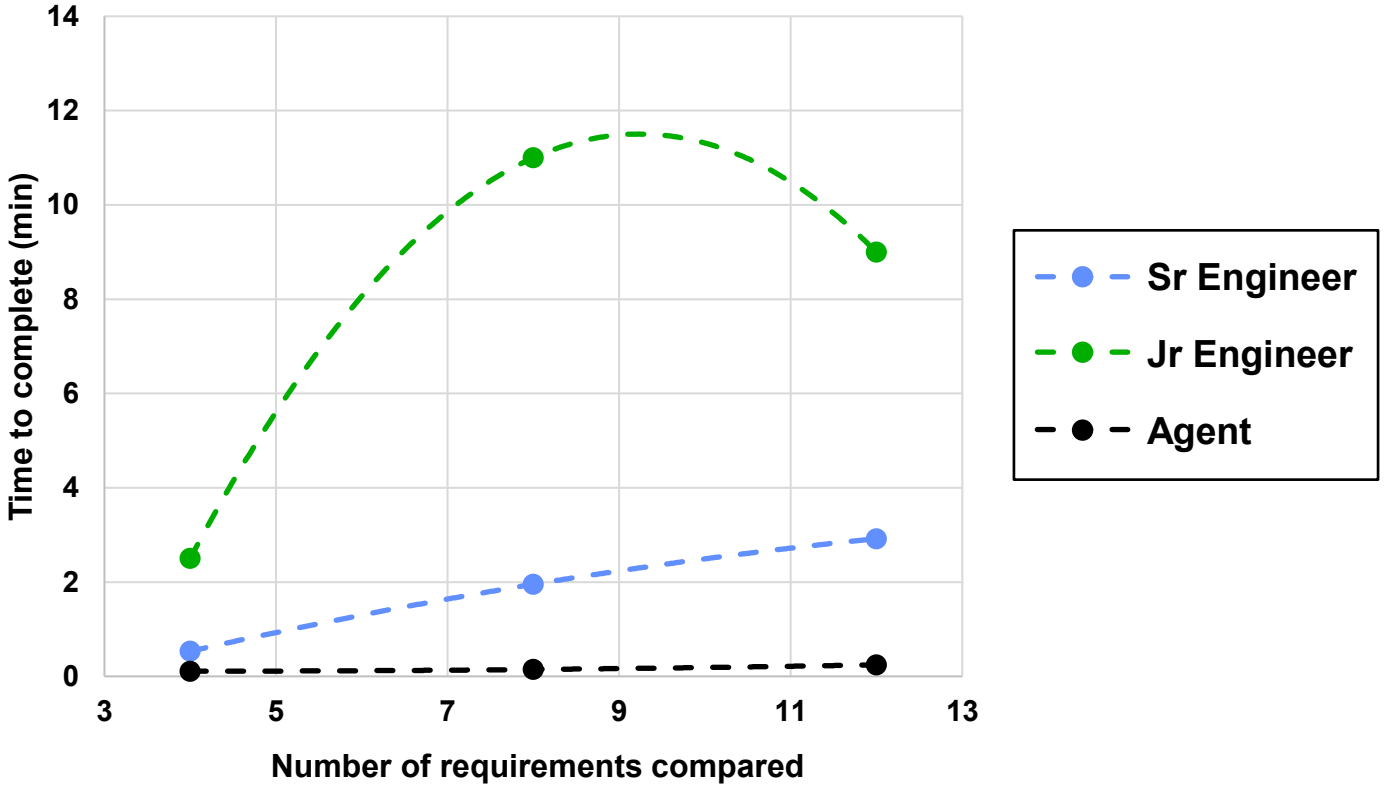
	Req ID	Requirement Name	Text
L0	REQ_001	system_pwr	The CubeSat shall generate, store, and distribute sufficient electrical power to support all mission operations for a minimum of 6 months.
	REQ_002	system_dataops	The CubeSat shall perform all data processing on-board.
L1	REQ_003	avgpwer	The EPS shall generate at least 20W average power in Low Earth Orbit (LEO).
	REQ_004	eclipseop	The electrical power system (EPS) shall store sufficient energy to operate the CubeSat in eclipse for at least 30 minutes.
	REQ_005	pwrdist	The EPS shall distribute regulated power to all subsystems within specified voltage tolerances.
	REQ_008	DataProcessingUnitRequirement	The system shall have a data processing unit.
L2	REQ_006	solar_eol	The solar array shall have a minimum end-of-life (EOL) power output of 22W in LEO.
	REQ_007	solareff_bol	The solar cells shall achieve at least 28% efficiency at beginning of life (BOL).
	REQ_009	RAMRequirement	The system shall have a computer chip with at least 8 GB of RAM
L3	REQ_010	NVIDIAChipRequirement	The data processing unit shall use an NVIDIA chip.
	REQ_011	SSDRequirement	The system shall use a solid-state drive (SSD) for storage.
	REQ_012	DPUStatesreq	The system shall power on and operate for 5s, then power down.

Expected Responses

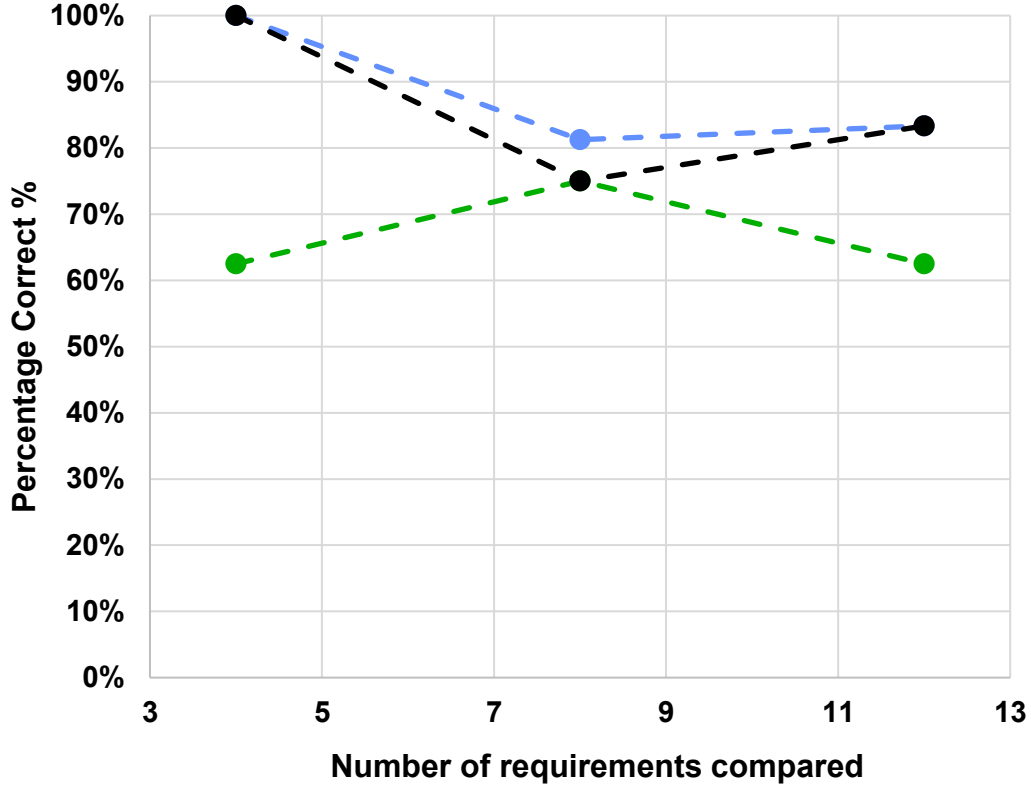
ID	Parent ID	Level	Req Name	Relationships (Satisfied By)
REQ_001		0	systempwr	Power Subsystem
REQ_002		0	system_dataops	Data Processing Unit
REQ_003	REQ_001	1	avgpwr	Power Subsystem
REQ_004	REQ_001	1	eclipseop	Power Subsystem: Batteries
REQ_005	REQ_001	1	pwrdist	Power Subsystem: ADC converter
REQ_006	REQ_003	2	solar_eol	Power Subsystem: Solar Array
REQ_007	REQ_003	2	solareff_bol	Power Subsystem: Solar Array
REQ_008	REQ_002	1	DataProcessingUnitRequirement	Data Processing Unit
REQ_009	REQ_008	3	RAMRequirement	Data Processing Unit
REQ_010	REQ_008	3	NVIDIAChipRequirement	Data Processing Unit: GPU
REQ_011	REQ_008	3	SSDRequirement	Data Processing Unit: SSD
REQ_012	REQ_008	3	DPUStatesreq	Data Processing Unit: PSU
				DPUstates *This is a behavior, must be a verified by relationship

Requirement Agent vs. Systems Engineers

Average Time



Average Percentage Correct



For the simplified problem, the agent performed comparably to engineers, and ~10x faster

Results – Requirement Agent

Orphan ID	Parent ID	Confidence	Rationale
REQ_001 (systempower)	**(no parent found)**	**NA**	There is no feasible parent requirement for this orphan because its target system points towards the top-level of the model (CubeSat).
REQ_008 (data_req)	**REQ_002 (system_dataops)**	**0.60**	The parent system_dataops matches structurally as it shares a subpart with the orphan requirement (CubeSat). Additionally, there is moderate textual overlap between the two requirements, with both mentioning data processing. However, no direct mention of Data Processing Unit is found in system_dataops, which makes the confidence score somewhat lower.
REQ_005 (powerdist)	**REQ_003 (avgpwer)**	**0.70**	The parent avgpwer satisfies structural containment evidence as it has a satisfies object whose part_definition contains a subpart electrical power system (EPS), which exactly matches the orphan's target_system. Additionally, avgpwer shares textual domain tokens with the orphan such as power, system and voltage tolerances. The parent avgpwer is more general in its wording compared to the orphan, making it a good match.



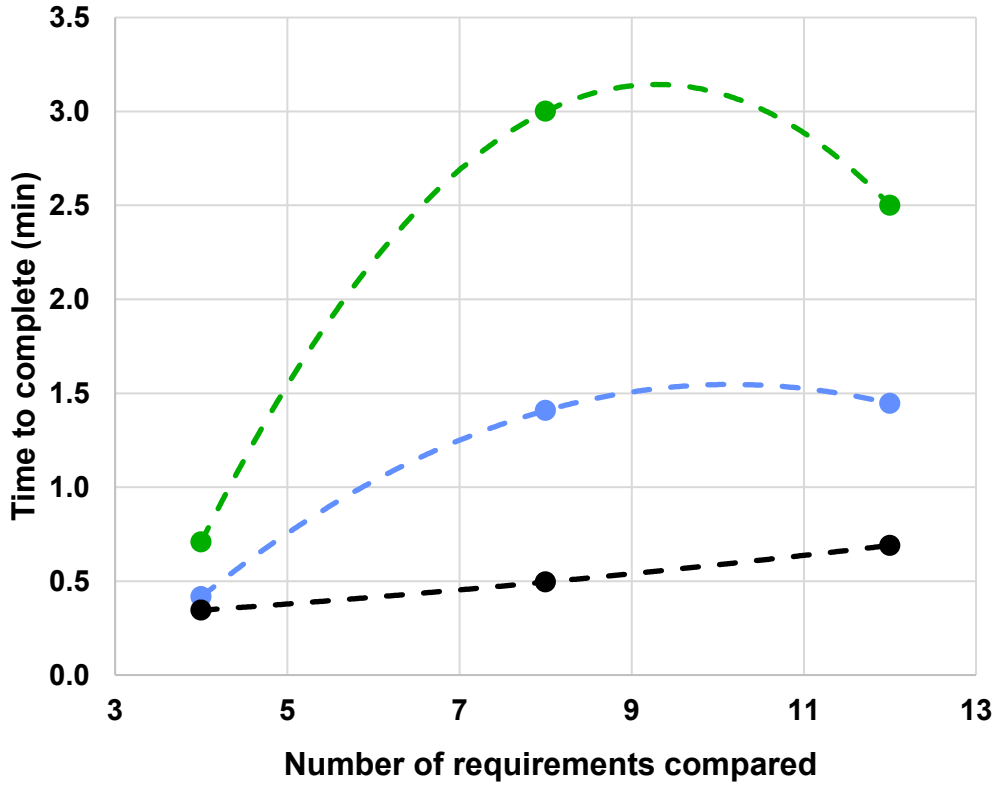


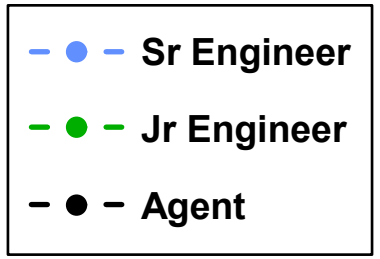


Agent rationale can provide powerful insight into model gaps and unintentional mistakes

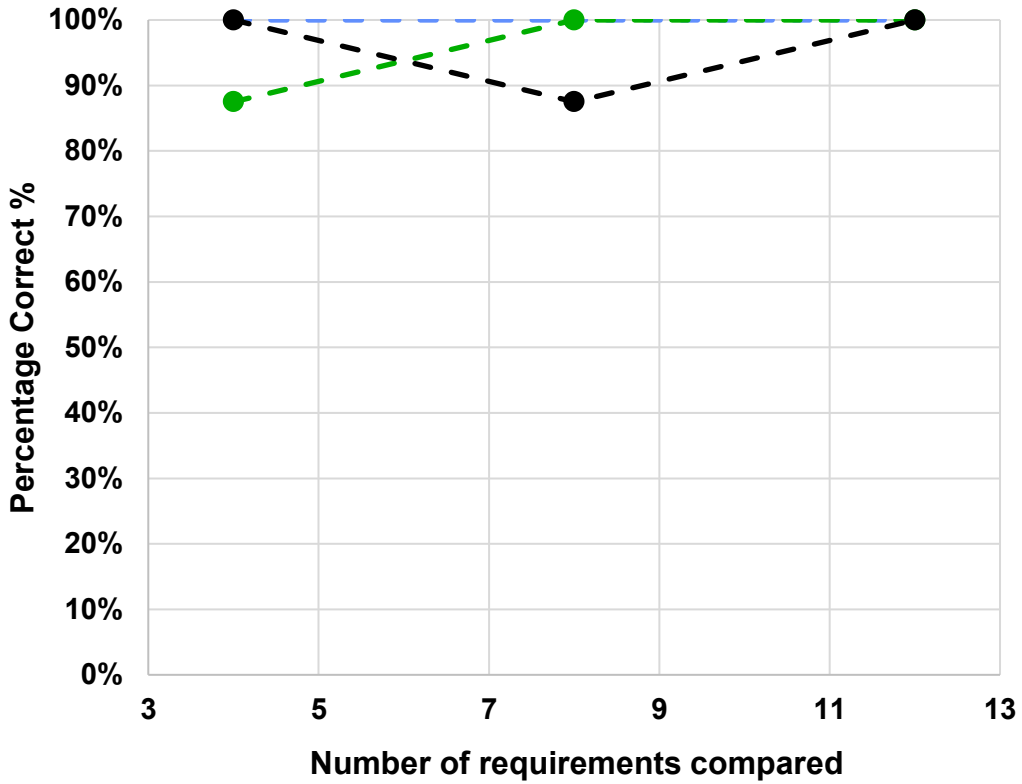
Relationship Agent vs. Systems Engineers

Average Time





Average Percentage Correct



The agent performed comparably to engineers, and at least twice as fast

Results - Relationship Agent

Req ID	Part	Relationship	Confidence	Rationale	
REQ_002	DataProcessingUnit	Satisfy	1.00	The DataProcessingUnit is designed to perform data processing tasks as indicated by its purpose and the presence of ports (an_in, cmd) for external data input/command. Its subparts include CPU, GPU, RAM, and storage which are typically used in data processing.	✓
REQ_002	FlightSysDef	Satisfy	0.40	The FlightSysDef is a part of the CubeSat's architecture, and while it doesn't explicitly have data processing capabilities, it could plausibly gain them through small extensions such as adding necessary ports for data input/output or integrating processing sub-parts. However, the confidence is moderate due to the absence of specific details about the FlightSysDef's current capabilities and the potential need for additional hardware or software.	✗
REQ_006	PowerGenerationSubsystem.solarpanels	Satisfy	0.70	The 'PowerGenerationSubsystem' includes the solarpanels subpart, which is a type of solar array. Given that there is a logical integration and clear connection between the solar panels and the requirement of having a minimum EOL power output, it is reasonable to conclude that the PowerGenerationSubsystem could satisfy the REQ-006.	✓
REQ_012	DPUStates	Verify	1.00	The behavior DPUStates directly simulates the system function described in requirements DPUStatesreq. The behavior specifies a sequence of states (on and off) that corresponds to the requirement's description of powering on and operating for 5s, then powering down.	✓

Outline

- Background & Approach
- Agentic Framework
- Engineers vs. Agents
- **Summary and Future Work**

Successes

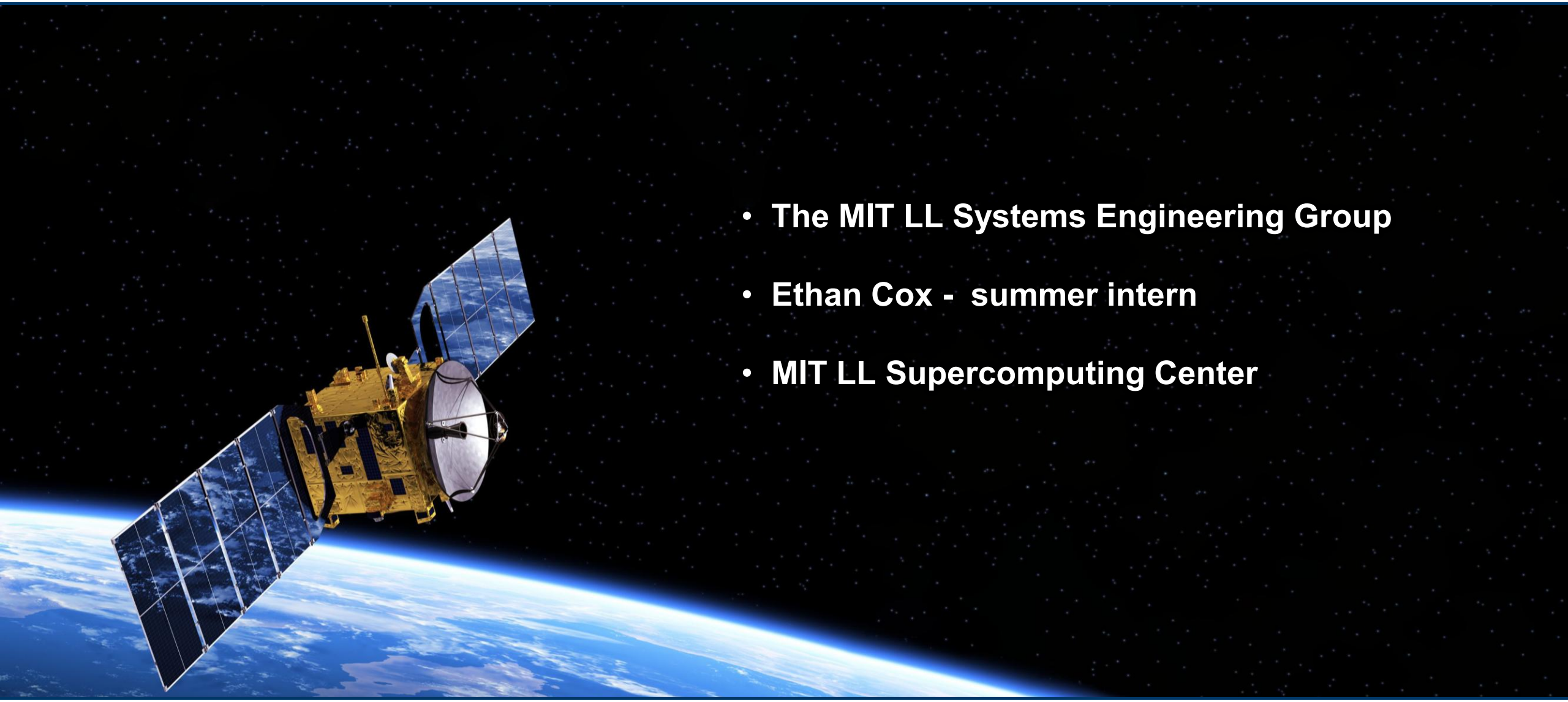
- Agents were successful in finding gaps in the provided SysMLv2 model
  - Reproduced SME-expected relationships in most cases
  - Some false-positives found, but often provided additional insights which improved model completeness
  - Can retrieve information quickly for efficient comparisons
- The agent framework is modular, scalable, and compliant with Laboratory data restrictions
  - New agents can be added easily
  - On-premise SLMs proved reliable, fast, and secure

This initial framework demonstrates a promising approach to using AI for systems engineering

Future Work

- Test the framework with larger, complex MBSE models
- Integrate framework into enterprise SysML v2 tools
- Leverage System Engineer's expertise in creating agents
  - Existing agents can become more sophisticated as more nuanced rules are identified
  - Additional agents can increase capability for validation tasks
    - Requirement quality
    - Identification of incompatible interfaces
    - Model generation recommendations
- New and better AI models rolling out can easily replace the SLM used
- Fine-tune SLMs-Improve SLM output by training on user data

Acknowledgements



- The MIT LL Systems Engineering Group
- Ethan Cox - summer intern
- MIT LL Supercomputing Center

Backup

Requirement Agent – Finds Potential Parent Requirements

Requirement Agent

Prompt	You are an expert SysMLv2 requirements reasoning agent.
Task	From the provided (pre-filtered) candidate parents, pick the single most appropriate parent for the given orphan requirements following the exact procedure below.
Procedure *summarized*	1. If the orphan is top-level (target system == model), consider only candidate parents whose target system == model. 2. Structural containment (strongest evidence) 3. Textual domain match (secondary evidence) 4. Penalize but do not eliminate same-target overlap 5. Tie-breaker 6. Final fallback
Important Rules	• You MUST choose the parent strictly from this list: {{valid parents}} (or return null). • Do NOT invent or return any parent not in that list. • Provide a concise rationale that explicitly cites the structural evidence (part/subpart match) or textual overlap used. • The rationale does not need to include confidence scores, but should explain the parent choice clearly.

LINCOLN LABORATORY
MASSACHUSETTS INSTITUTE OF TECHNOLOGY

Relationship Agent – Finds Potential Relationships

Relationship Agent

Prompt	You are a SysMLv2 requirements hypothetical reasoning agent.
Task	Decide whether a system object (a part definition) could reasonably satisfy a given requirement within the scope of the current architecture.
Procedure	- If the object already has the necessary role or immediate attributes to satisfy the requirement, answer “yes”. - If the object could plausibly gain the necessary role or attributes through small, natural extensions (e.g., adding a part, assigning an attribute that fits the subsystem’s role), answer “yes”. - If the requirement is unrelated to the object’s purpose, structure, or capabilities, answer “no”. - If the requirements could be satisfied only by unreasonable changes or with no clear connection, answer “no”.
Important Rules	- If the object could logically integrate or provide the capability the requirement describes -> “yes”. - If the requirement could be satisfied only by unreasonable changes or with no clear connection -> “no”. - If the object could satisfy the requirement with a sub-part extension, but the domain match is weak -> “no”. - Prefer “no” in most cases unless there is a clear, justifiable link.